Tecnicatura Superior en Desarrollo de Software

Módulo 10 — `Blazor Hybrid`: Componentes Razor dentro de una Aplicación de Escritorio

Asignatura: Programación II | Instituto de Educación Superior N.º 9007 Dr. Salvador Calafat | Ciclo Lectivo 2026

¿Qué es Blazor Hybrid?

Blazor Hybrid	Componentes Razor que se muestran dentro de una aplicación de escritorio	Capa: Frameworks y Drivers (el host)

Un componente Razor es una pieza de interfaz escrita con HTML y C# (un archivo `.razor`). Normalmente se muestra en una página web. **Blazor Hybrid** permite mostrar esos mismos componentes dentro de una aplicación de escritorio: la ventana es de WPF, pero el contenido del formulario lo dibuja un navegador embebido (**WebView2**) a partir de un componente Razor. No hay servidor web: el código C# corre dentro del mismo programa.

Piezas que intervienen:

- El control **BlazorWebView** (en la ventana)
- Una **página host** (`index.html`)
- Un **componente raíz** (el que se muestra)
- El **contenedor** de dependencias del host

En los módulos anteriores la View era un componente Razor que nadie mostraba. El host del Módulo 9 era una ventana sin pantalla de negocio. Blazor Hybrid es la pieza que une las dos cosas: la ventana del host pasa a contener el componente.

Analogía del mundo real

Pensá en un televisor inteligente dentro de un mueble. El **mueble** es la ventana de la aplicación de escritorio: tiene su marco, su tamaño y es lo que ponés en la sala. La **pantalla** es el navegador embebido (WebView2): un rectángulo dentro del mueble donde se ve el contenido. Y el **programa que se está reproduciendo** es el componente Razor.

Al televisor no le importa qué programa pasa: si mañana cambiás el programa, no tocás el mueble. Y al programa no le importa si lo ves en ese televisor o en otro. Esa independencia es justamente lo que buscamos: el componente no sabe que vive dentro de una aplicación de escritorio, y la ventana no sabe qué hace el componente.

Características esenciales

1. No hay servidor ni internet de por medio

Aunque se vea como una página web, todo ocurre dentro del mismo proceso del programa. El código C# del componente y el del resto de la aplicación son el mismo código, en la misma memoria. Por eso un componente puede usar directamente un ViewModel o un caso de uso, sin llamadas de red.

2. El componente no cambia respecto de uno web

Un componente Razor escrito para una web funciona igual en Blazor Hybrid. Lo que cambia es dónde se aloja, y eso es responsabilidad del host. Es lo que permite reutilizar la View del Módulo 7 sin modificarla.

3. Comparte el contenedor de dependencias del host

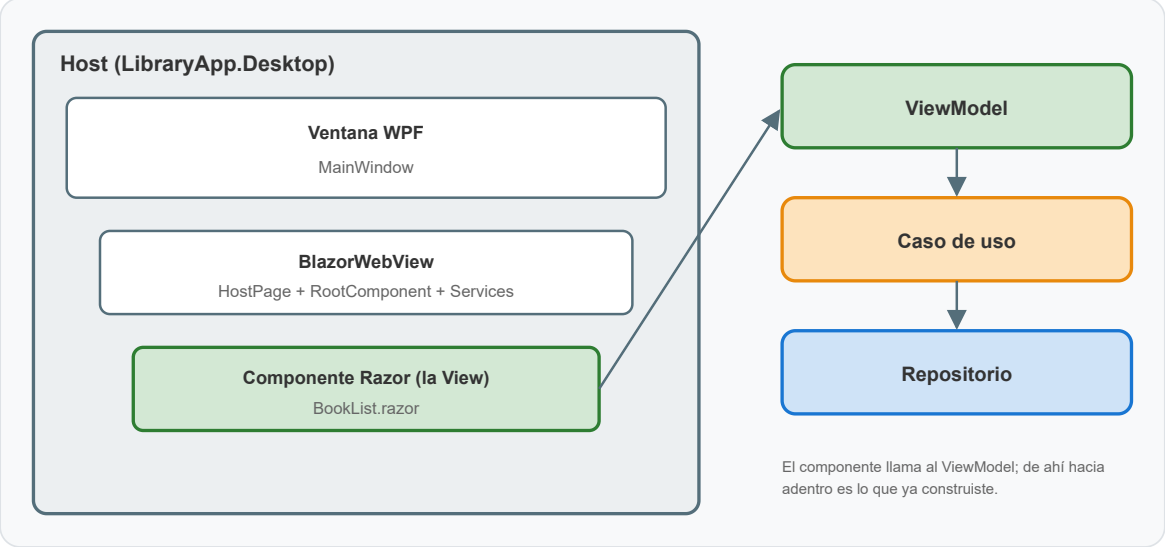
Al `BlazorWebView` se le entrega el mismo contenedor que armó el composition root. Así los componentes reciben sus ViewModels, casos de uso y repositorios ya conectados, sin que nadie los construya con `new`.

4. Depende del runtime de WebView2

WebView2 es el motor de Microsoft Edge. Windows 10 y 11 actualizados ya lo traen. En una máquina que no lo tenga, el control no puede mostrar el contenido.

¿Cómo encaja en la arquitectura?

El `BlazorWebView` es un detalle del host: pertenece al anillo más externo, igual que WPF. La pantalla (la View) sigue siendo una pieza de InterfaceAdapter, y de ahí hacia adentro no cambia nada. El host solo tiene que saber *dónde* mostrarla.



Fijate en lo que **no** aparece: el host no llama al caso de uso ni al repositorio. Solo muestra el componente. Toda la lógica sigue donde estaba.

Tres formas de alojar Blazor

Blazor es un solo modelo de componentes que puede ejecutarse en distintos lugares. Conviene distinguirlos porque suelen confundirse:

Modelo	Dónde corre el código C#	Qué se necesita
Blazor Server	En un servidor; el navegador solo muestra la página	Un servidor web y conexión permanente
Blazor WebAssembly	Dentro del navegador del usuario	Un servidor que entregue los archivos
Blazor Hybrid	Dentro de una aplicación nativa (WPF en este curso)	Un navegador embebido (WebView2); ningún servidor web

En este curso usamos Blazor Hybrid porque el sistema es una aplicación de escritorio que habla directamente con SQL Server. No hay controladores ni `Program.cs` de aplicación web en ningún punto.

Las piezas del BlazorWebView

La página host

Es un `index.html` casi vacío. Tiene un elemento reservado para el componente y el script que conecta la página con el código C#:

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="utf-8" />
    <base href="/" />
</head>
<body>
    <div id="app">Cargando...</div>
    <script src="_framework/blazor.webview.js"></script>
</body>
</html>
```

El archivo `blazor.webview.js` no lo escribís vos: lo aporta el paquete del `BlazorWebView`. Por eso el proyecto del host necesita el SDK Razor (el que publica esos archivos). El texto "Cargando..." se ve un instante y después lo reemplaza el componente.

El control y su componente raíz

En el XAML de la ventana se declara el control, qué página usa y qué componente se muestra en cuál elemento:

```
<blazor:BlazorWebView HostPage="wwwroot\index.html">
    <blazor:BlazorWebView.RootComponents>
        <blazor:RootComponent Selector="#app" ComponentType="{x:Type views:BookList}" />
    </blazor:BlazorWebView.RootComponents>
</blazor:BlazorWebView>
```

`Selector="#app"` es el elemento reservado del `index.html` y `ComponentType` el componente que se dibuja ahí. Los prefijos `blazor:` y `views:` son alias de espacios de nombres que se declaran al principio del archivo XAML.

Entregarle el contenedor

El control no sabe de dónde sacar las dependencias de los componentes hasta que se lo decís. Se le entrega el contenedor del host en el code-behind de la ventana:

```
public MainWindow(IServiceProvider services)
{
    InitializeComponent();

    // los componentes Razor resuelven sus dependencias del mismo contenedor del host
    BlazorView.Services = services;
}
```

Y en el composition root se registran los servicios internos del control con una sola línea. Los componentes **no** se registran en el contenedor: los crea Blazor.

```
// servicios que necesita el BlazorWebView para mostrar componentes Razor
services.AddWpfBlazorWebView();
```

Dependencias y tiempos de vida dentro de un componente

En el Módulo 9 viste que en una aplicación de escritorio nadie abre scopes por vos. En Blazor Hybrid pasa algo parecido: el contenedor que recibe el `BlazorWebView` dura lo que dura la ventana, así que un servicio *Scoped* pedido directamente con `@inject` se comporta, en la práctica, como si fuera de vida larga: se comparte entre todas las pantallas y conserva su estado.

Para un ViewModel de formulario eso es un problema: no querés que el texto de un formulario quede guardado cuando el usuario sale y vuelve. La solución es que el componente **cree su propio scope**. Eso hace la clase base `OwningComponentBase<T>`:

```
1  @* BookList.razor *@
2  @inherits OwningComponentBase<BookListViewModel>
3
4  <button @onclick="Service.LoadAsync">Cargar</button>
```

Cómo funciona: `OwningComponentBase<T>` abre un scope propio para el componente, resuelve en él un `T` y lo expone como `Service`. Cuando el componente se destruye, cierra el scope y libera todo lo que se creó en él. Cada pantalla recibe su ViewModel y sus servicios Scoped nuevos, y los descarta al salir.

> **Qué se registra y qué no:** se registra el ViewModel (`AddScoped<BookListViewModel>`), el caso de uso, el presenter y el repositorio. No se registra el componente `BookList`.

Errores comunes

ERROR COMUN

Inyectar el acceso a datos directamente en el componente:

```
@inject LibraryDbContext Db

<button @onclick="Save">Guardar</button>
```

La pantalla queda atada a Entity Framework y se saltea los casos de uso y sus reglas.

BUENA PRACTICA

El componente solo conoce a su ViewModel:

```
@inherits OwningComponentBase<BookListViewModel>

<button @onclick="Service.SaveAsync">Guardar</button>
```

ERROR COMUN

Poner lógica de la pantalla en el code-behind de la ventana del host:

```
private async void OnSaveClick(object sender, RoutedEventArgs e)
{
    // el host no debe validar ni decidir reglas
    if (_title.Length < 2) return;
}
```

El host es solo el marco: la lógica de la pantalla va en el ViewModel y las reglas, en el caso de uso.

BUENA PRACTICA

El host solo muestra el componente y le entrega el contenedor:

```
BlazorView.Services = services;
```

Cómo se manifiesta cada olvido: sin `AddWpfBlazorWebView()` el programa falla al abrir con un mensaje que pide llamarlo; sin asignar `Services`, la zona del formulario queda en blanco y no hay ningún error; y sin el script `blazor.webview.js` en la página host, se queda para siempre el texto "Cargando...". Los dos últimos no avisan: conviene conocerlos.

Resumen

Blazor Hybrid muestra componentes Razor dentro de una aplicación de escritorio.

El host tiene un `BlazorWebView` que usa una página `index.html`, un componente raíz y el contenedor de dependencias del composition root. El componente llama a su ViewModel y de ahí hacia adentro es el mismo código de siempre. Cada componente de formulario usa `OwningComponentBase<T>` para tener su propio scope.

Resultado: agregar o cambiar una pantalla es tocar el host y la View, no las reglas del negocio.